IoT-Based Weather Station

ESP32, DHT22 and ThingSpeak Cloud Dashboard

Prepared by	Haresh Selvakumar
Institution	VIT Chennai
Department	Electronics and Communication Engineering

Simulation platform	Velxio
Cloud channel	3499317 - IoT Weather Station - Haresh
Report date	19 September 2026

Abstract

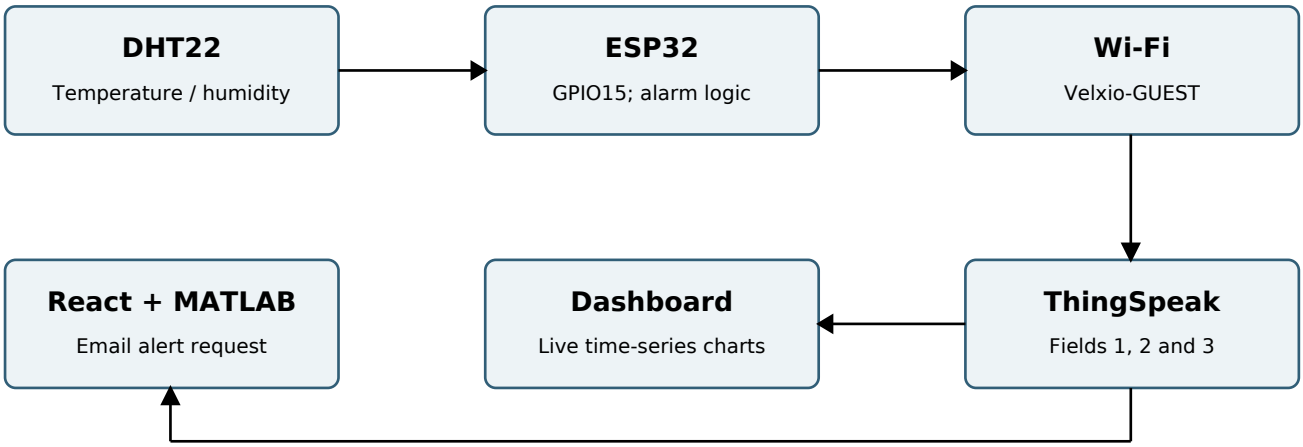
This project implements an ESP32 weather-monitoring system with a DHT22 sensor and a ThingSpeak dashboard. Temperature and relative humidity are acquired in Velxio and transmitted to a private cloud channel. An alarm flag is calculated when temperature exceeds 35 C or humidity exceeds 80%. A saved ThingSpeak React rule invokes MATLAB Analysis to request an email when the alarm condition is first met.

Results overview

The firmware acquired valid simulated readings and transmitted them to ThingSpeak. The captured channel contains 52 entries: 49 normal sensor uploads and three controlled cloud test records. Cloud readback confirms both threshold conditions and the intervening normal reset. Received email screenshots confirm temperature and humidity alerts matching the test records.

Validation combines simulated DHT22 acquisition with separately injected cloud test inputs. The readings are test data, not measurements of the surrounding environment.

1. System architecture



The DHT22 provides digital temperature and humidity values to GPIO15. The ESP32 validates each reading, evaluates the thresholds and sends all three fields in one update. ThingSpeak timestamps the readings and refreshes the dashboard. React evaluates Field 3 independently of the chart display.

2. Sensor interfacing

Signal / component	Connection or setting
DHT22 VCC / GND	ESP32 3V3 / GND
DHT22 SDA	ESP32 GPIO15
DHT22 NC	Unconnected
Data pull-up	10 kohm between SDA and 3V3 in the imported diagram
Libraries	DHT sensor library 1.4.7; Adafruit Unified Sensor 1.1.15
Serial monitor	115200 baud

The sketch calls dht.begin() during setup. Each acquisition uses readTemperature() and readHumidity(). NaN values and readings outside -40 to 80 C or 0 to 100% RH are rejected. The initial delay is 2.1 seconds. Subsequent cycles wait at least 20 simulated seconds after the preceding cycle finishes.

3. Cloud upload and field mapping

Field	Quantity	Unit / meaning
1	Temperature	Degrees Celsius
2	Relative humidity	Percent
3	Alarm state	1 if T > 35 C OR RH > 80%; otherwise 0

The ESP32 connects to the virtual Velxio-GUEST access point and sends an HTTP POST to api.thingspeak.com/update. The form body contains the channel write key and field1, field2 and field3. The program accepts an upload only when HTTP status is 200 and the response contains a positive entry ID; a response of 0 is treated as a failed update.

The submitted source contains a key placeholder. Enter the write key privately in the working simulator. The educational implementation uses HTTP, so the write key is unencrypted in transit; a deployed device should use certificate-validated HTTPS.

4. Cloud visualization and observed results

The channel has two dynamically refreshed line charts. Both use time on the horizontal axis. Field 1 is labelled Temperature (C); Field 2 is labelled Relative humidity (%). The captured dashboard reports 49 entries. The recorded normal operating condition is 28.0 C and 60.0% RH, matching the simulated sensor input.

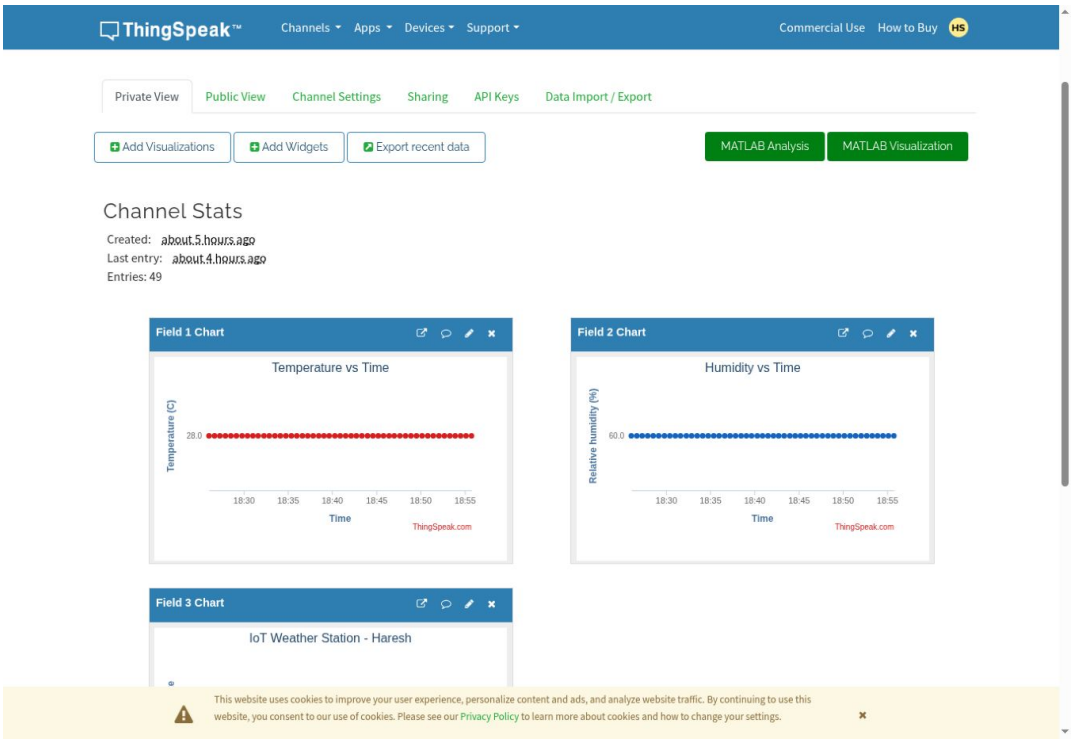


Figure 1. Actual ThingSpeak dashboard: 49 stored entries, temperature 28.0 C and humidity 60.0%.

5. Email alert mechanism

The saved MATLAB Analysis, Weather Station Email Alert (ID 424064), reads the latest values from channel 3499317. When the third field is 1, it calls the ThingSpeak Alerts API over HTTPS. The message includes temperature, humidity and the configured limits. Account credentials and API keys are excluded from the submitted files.

React 137446, Weather Station Threshold Alert, uses Numeric conditions, tests On Data Insertion, and runs the analysis when Field 3 equals 1. The option Run action only the first time the condition is met suppresses repeated requests while the alarm remains active. Both readings must return to normal in an accepted upload before a new abnormal episode can trigger another request.

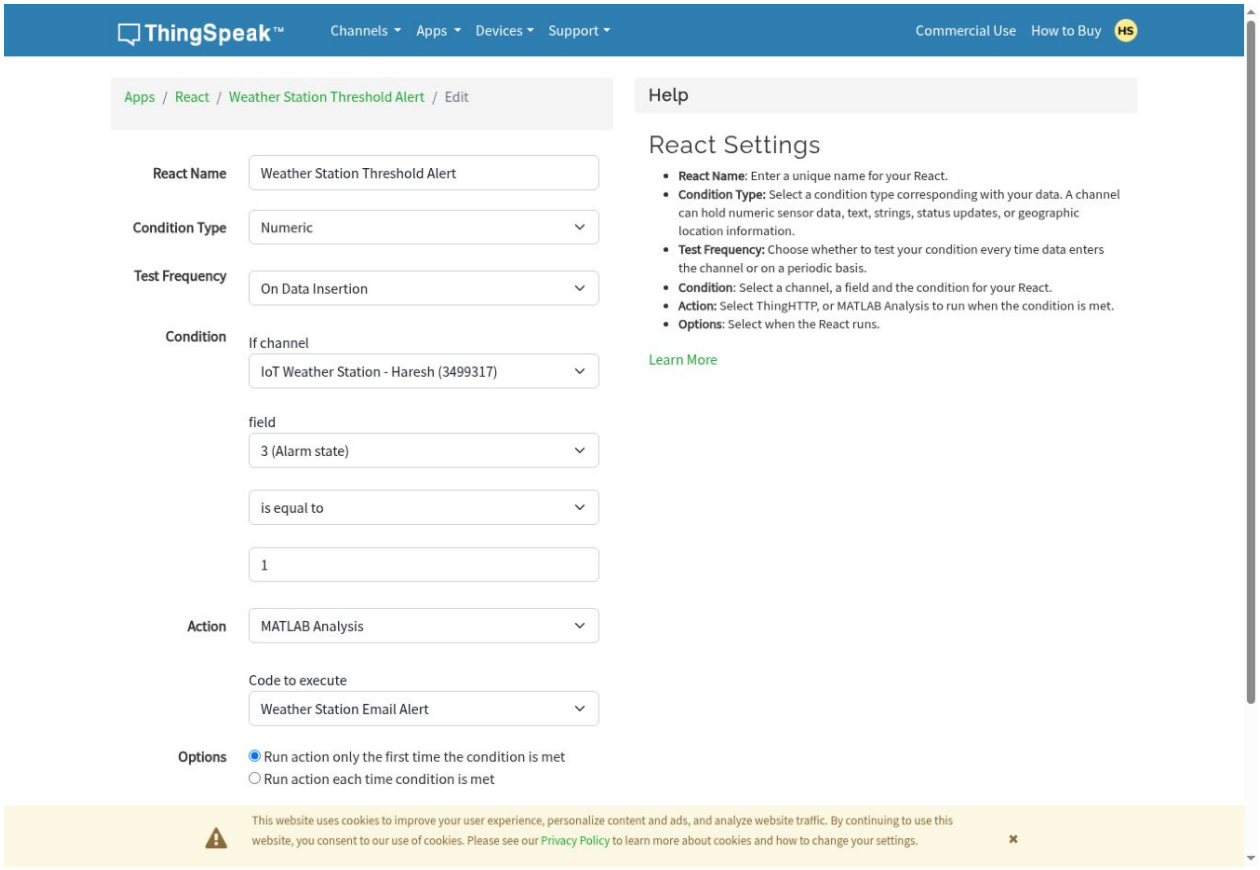


Figure 2. Saved React rule: Field 3 equals 1, evaluated on data insertion, linked to the email analysis.

The first-occurrence setting reduces repeated notifications during a sustained alarm. This implementation does not retry rejected email requests. Received-message evidence for both test conditions is presented in Section 9.

6. Validation record

Check	Observed outcome
ESP32 compilation	Successful in Velxio using the Adafruit DHT driver
Sensor acquisition	28.0 C, 60.0% RH, alarm 0 observed in the imported circuit
Cloud storage	52 entries: 49 sensor uploads plus three cloud test records
Temperature and humidity graphs	Normal readings and separate threshold-test peaks displayed
React configuration	Saved: Numeric; Field 3 = 1; On Data Insertion; first occurrence
Temperature > 35 C cloud test	Readback: 40 C, 60% RH, alarm 1
Humidity > 80% cloud test	Readback: 28 C, 85% RH, alarm 1
Received alert emails	Confirmed by supplied inbox screenshots for both conditions

Validation scope

The sensor-to-cloud path was demonstrated at 28 C and 60% RH. The cloud-to-email path was tested separately using MATLAB-written records for temperature and humidity alarms. Both received emails match those records. A continuous abnormal DHT22-to-email run was not captured; the evidence establishes these two stages separately.

Conclusion

The weather station demonstrates digital sensor acquisition, Wi-Fi cloud upload, live visualization and threshold-based email notification. A temperature of 40 C triggered the temperature alert, and 85% RH triggered the humidity alert. A normal record between these tests reset the alarm condition. The implementation meets the functional objectives within the staged validation described above.

Project access

Private dashboard:
https://thingspeak.mathworks.com/channels/3499317/private_show
Access requires the channel owner to sign in.

7. Cloud commissioning results

The final verification used MATLAB Analysis 424093 to write three controlled inputs to the existing channel. Each write was separated by more than the channel update interval. The latest three records were then independently read back using thingSpeakRead. These are cloud integration tests, separate from the earlier 49 ESP32 uploads.

Test	Temperature	Humidity	Alarm
High temperature	40 C	60%	1
Normal reset	28 C	60%	0
High humidity	28 C	85%	1

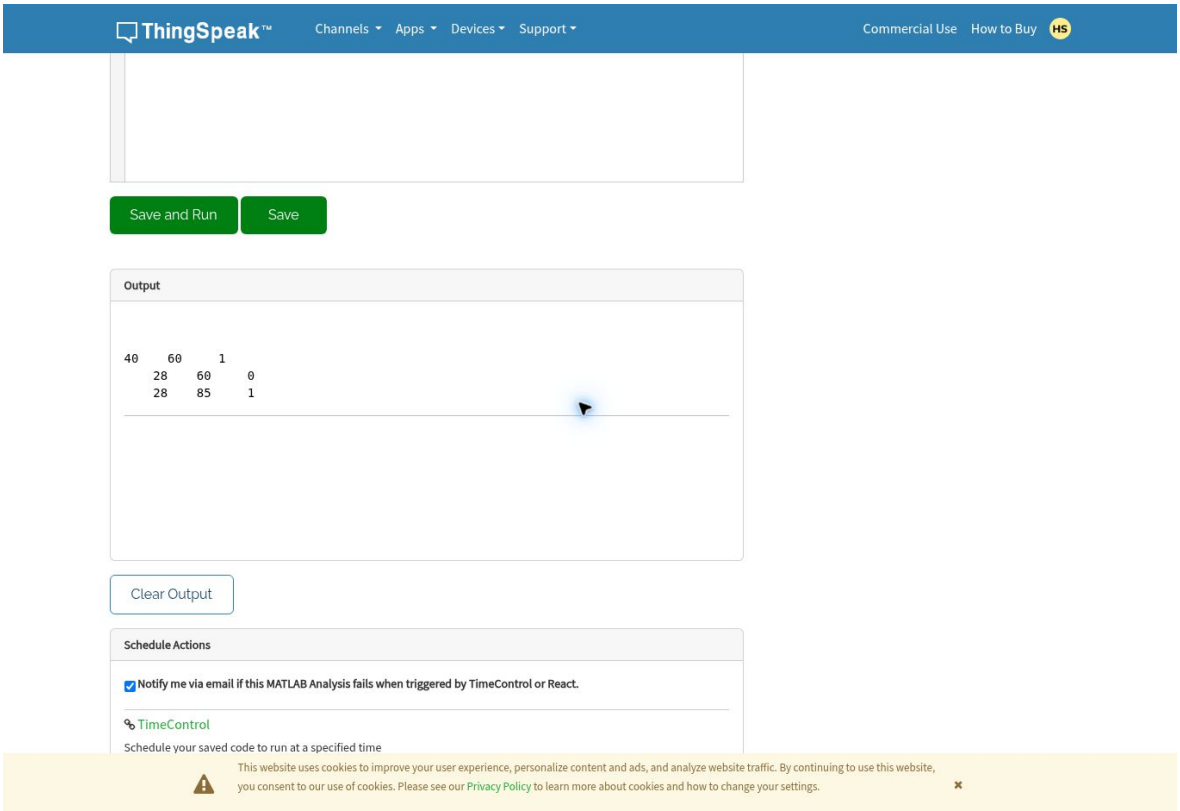


Figure 3. Actual MATLAB readback of the three stored records. Columns: temperature (C), relative humidity (%) and alarm state.

The normal record between the alarms resets the first-occurrence condition in React. Readback confirms storage of both abnormal states. The received messages in Section 9 confirm the corresponding email notifications. The commissioning analysis was left as a read-only audit, with no scheduled action.

8. Dashboard after threshold tests

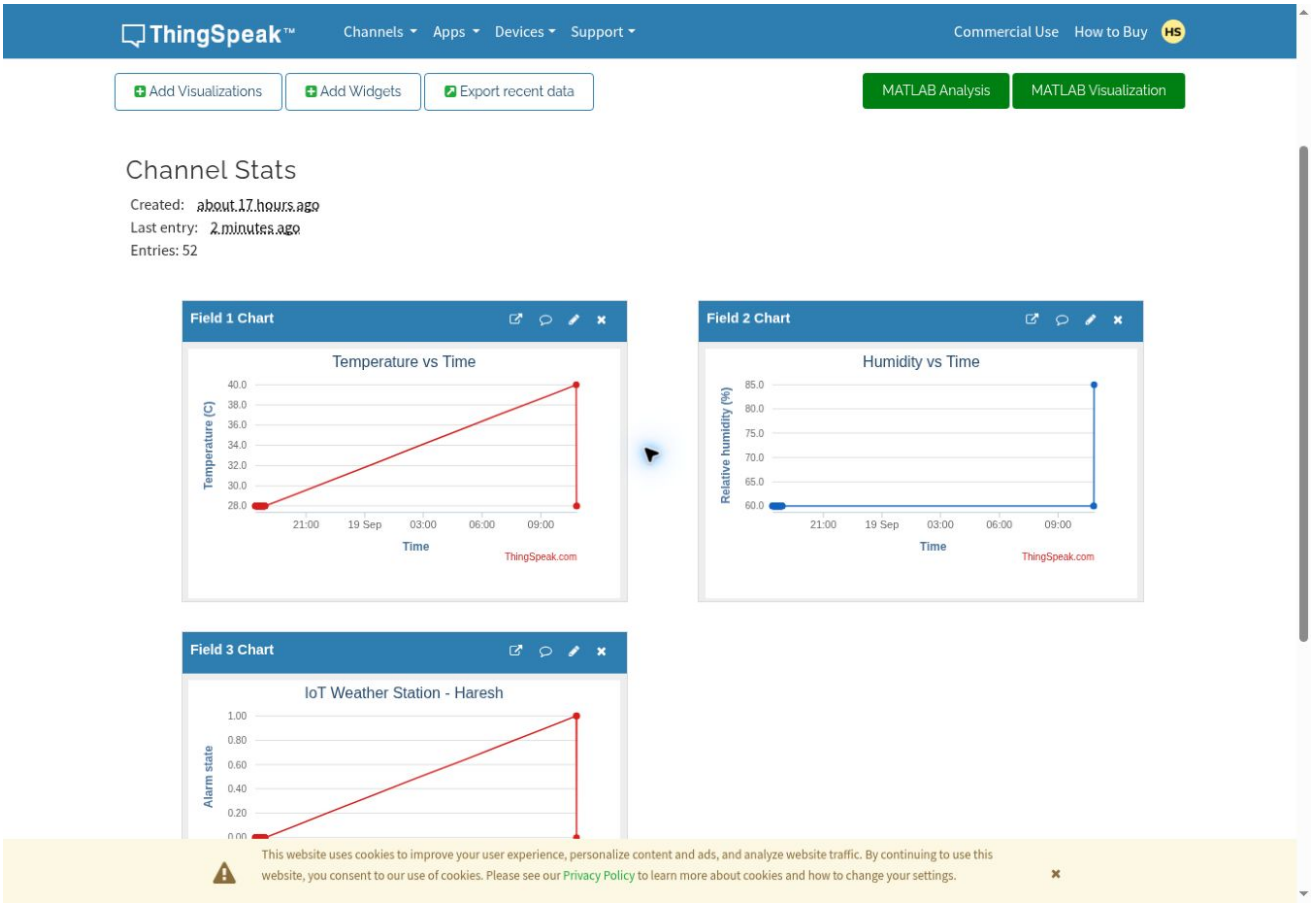


Figure 4. Actual dashboard after commissioning: 52 entries, temperature peak 40 C and humidity peak 85%. The long line segments bridge the gap between simulation and commissioning sessions; they are not continuous environmental measurements.

The original normal-condition graphs are retained for comparison. Temperature rises to 40 C in the first cloud test; after the normal reset, humidity rises to 85%. These discrete commissioning records explain the peaks in the displayed series.

9. Received email alerts

The supplied inbox screenshots document two received ThingSpeak alerts on 19 September 2026. Each message contains the tested reading and the limits T > 35 C or RH > 80%. The temperature test keeps humidity normal; the humidity test keeps temperature normal, demonstrating the two independent branches of the OR condition.

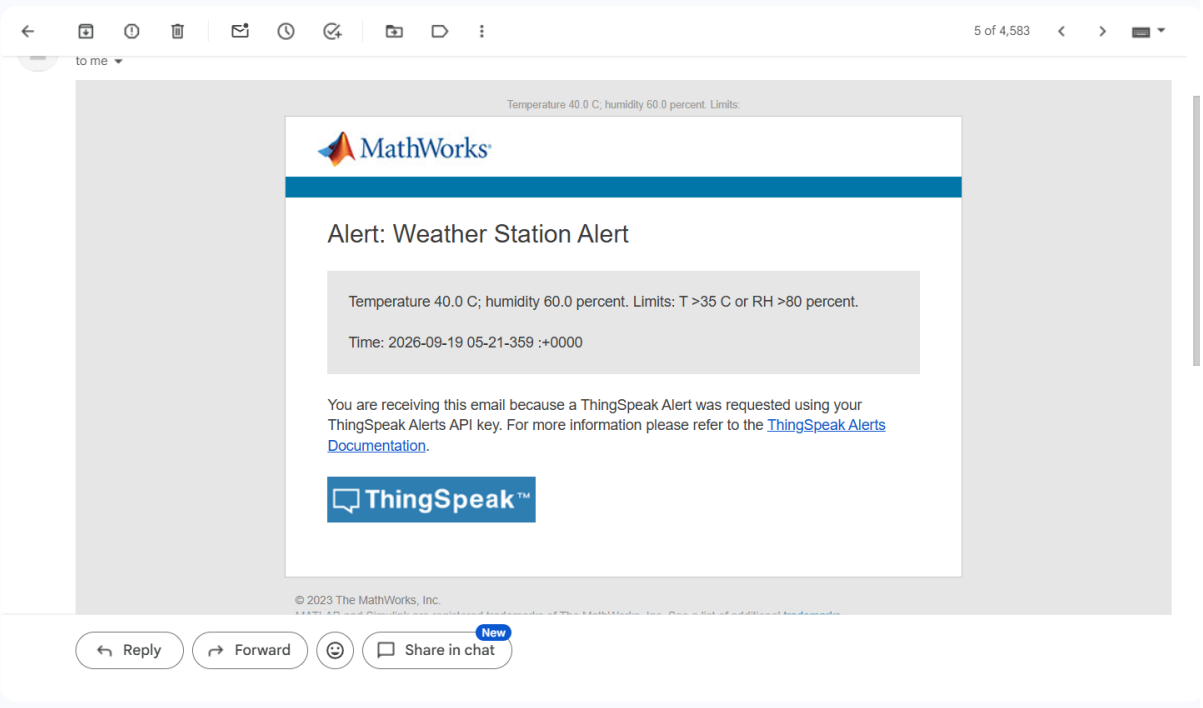


Figure 5. Received temperature alert: 40.0 C and 60.0% RH. Temperature exceeds the 35 C threshold.

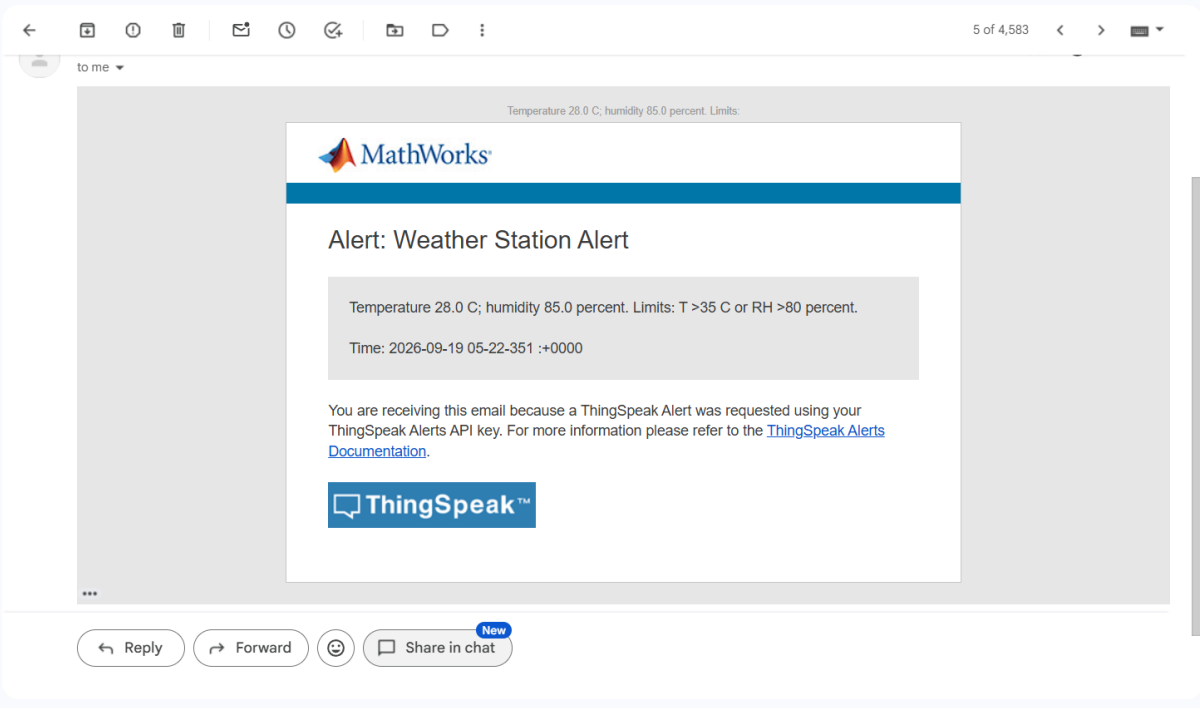


Figure 6. Received humidity alert: 28.0 C and 85.0% RH. Humidity exceeds the 80% threshold.

Appendix A. Reproducing the simulation

Open velxio.dev/editor and import Velxio_Import.zip. The package contains sketch.ino, diagram.json and libraries.txt at the ZIP root. After import, open Libraries and add DHT sensor library by Adafruit and Adafruit Unified Sensor. The imported library list may not automatically install these dependencies.

The circuit uses ESP32 DevKit V1, DHT22 and a 10 kohm pull-up. Compile and run the sketch. A placeholder key deliberately enables sensor-only demonstration mode. Use the write key from channel 3499317 for cloud tests. Click the DHT22 to adjust its virtual temperature and humidity.

Appendix B. Submitted files

File	Purpose
Project_Report.pdf	Architecture, implementation, results and evidence
sketch.ino	ESP32 acquisition, threshold and cloud firmware
weather_alert.m	MATLAB alert analysis; keys omitted
cloud_commissioning.m	Controlled cloud test procedure; keys omitted
diagram.json	Portable circuit and wiring
libraries.txt	Required sensor dependencies
Velxio_Import.zip	Source/circuit package for simulator import
Screenshots/	Original screenshots of the configured system

The report, source code and original evidence images are included in the project folder. API keys are represented by placeholders in distributed code; the configured cloud analysis retains its private account settings.

Appendix C. Circuit evidence

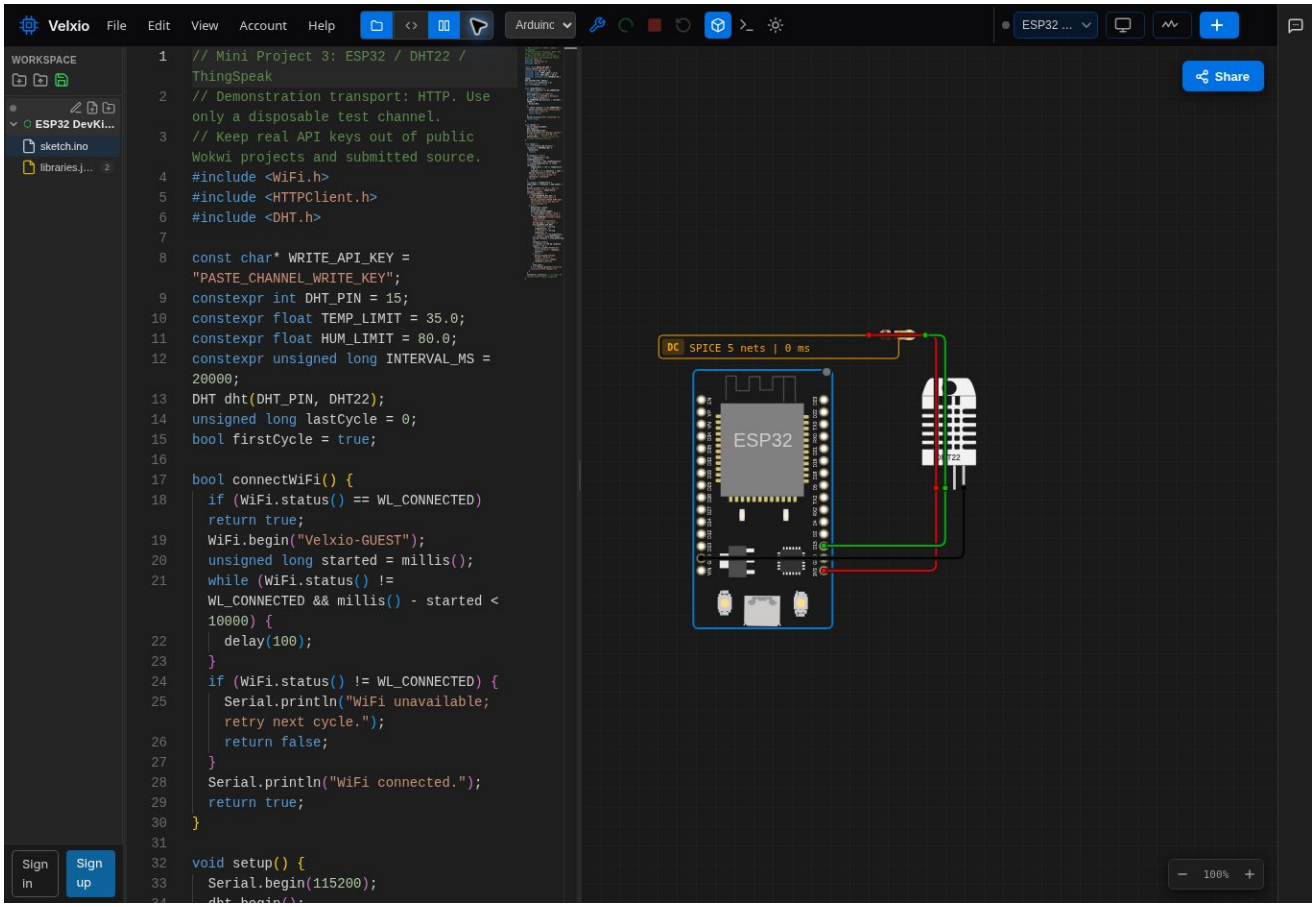


Figure 7. Imported ESP32 and DHT22 circuit in Velxio. The displayed source contains a key placeholder. The circuit screenshot documents the simulated wiring.

References

Project brief: 4844301-IOT_Mini_Project.pdf, supplied assignment

Velxio simulator: https://velxio.dev/

ThingSpeak Write Data: https://www.mathworks.com/help/thingspeak/writedata.html

ThingSpeak React: https://www.mathworks.com/help/thingspeak/react-app.html

ThingSpeak Alerts API: https://www.mathworks.com/help/thingspeak/sendalert.html

Appendix D. ESP32 firmware

Complete submitted sketch. Configure the private channel write key before cloud operation. The firmware rejects invalid readings and reports upload success only for a positive ThingSpeak entry ID.

```
// Mini Project 3: ESP32 / DHT22 / ThingSpeak
// Demonstration transport: HTTP. Use only a disposable test channel.
// Keep real API keys out of public Wokwi projects and submitted source.
#include <WiFi.h>
#include <HTTPClient.h>
#include <DHT.h>

const char* WRITE_API_KEY = "PASTE_CHANNEL_WRITE_KEY";
constexpr int DHT_PIN = 15;
constexpr float TEMP_LIMIT = 35.0;
constexpr float HUM_LIMIT = 80.0;
constexpr unsigned long INTERVAL_MS = 20000;
DHT dht(DHT_PIN, DHT22);
unsigned long lastCycle = 0;
bool firstCycle = true;

bool connectWiFi() {
  if (WiFi.status() == WL_CONNECTED) return true;
  WiFi.begin("Velxio-GUEST");
  unsigned long started = millis();
  while (WiFi.status() != WL_CONNECTED && millis() - started < 10000) {
    delay(100);
  }
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("WiFi unavailable; retry next cycle.");
    return false;
  }
  Serial.println("WiFi connected.");
  return true;
}

void setup() {
  Serial.begin(115200);
  dht.begin();
  WiFi.mode(WIFI_STA);
  Serial.println("IoT Weather Station | T limit 35 C | RH limit 80 %");
  delay(2100); // DHT22 startup and minimum sampling interval
}

void loop() {
  if (!firstCycle && millis() - lastCycle < INTERVAL_MS) {
    delay(10);
    return;
  }
  firstCycle = false;
  float temperature = dht.readTemperature();
```

Appendix D. ESP32 firmware (continued)

```
  float humidity = dht.readHumidity();
  if (isnan(temperature) || isnan(humidity) ||
      temperature < -40 || temperature > 80 ||
      humidity < 0 || humidity > 100) {
    Serial.println("Invalid DHT22 reading; no cloud upload.");
    lastCycle = millis();
    return;
  }
  int alarm = (temperature > TEMP_LIMIT || humidity > HUM_LIMIT) ? 1 : 0;
  Serial.printf("T=%.1f C | RH=%.1f %% | alarm=%d\n", temperature, humidity, alarm);
  if (connectWiFi()) {
    if (String(WRITE_API_KEY) == "PASTE_CHANNEL_WRITE_KEY") {
      Serial.println("SENSOR DEMO ONLY: add channel Write API Key for cloud uploads.");
    } else {
      WiFiClient client;
      HTTPClient http;
      http.setTimeout(10000);
      if (http.begin(client, "http://api.thingspeak.com/update")) {
        http.addHeader("Content-Type", "application/x-www-form-urlencoded");
        String body = "api_key=" + String(WRITE_API_KEY)
          + "&field1=" + String(temperature, 1)
          + "&field2=" + String(humidity, 1)
          + "&field3=" + String(alarm);
        int status = http.POST(body);
        String response = http.getString();
        response.trim();
        if (status == 200 && response.toInt() > 0) {
          Serial.printf("UPLOAD OK | entry_id=%ld\n", response.toInt());
        } else {
          Serial.printf("UPLOAD FAILED | HTTP=%d | response=%s\n", status, response.c_str());
        }
        http.end();
      } else Serial.println("Could not initialize HTTP request.");
    }
  }
  lastCycle = millis(); // at least 20 seconds AFTER request completes
}
```

Appendix E. MATLAB email analysis

This analysis is invoked by React when the uploaded alarm field is 1. The channel read key and Alerts API key must be configured privately in ThingSpeak.

```
% ThingSpeak MATLAB Analysis: Weather Station Email Alert
% Enter keys privately in ThingSpeak before running.
v = thingSpeakRead(3499317, 'Fields', [1 2 3], ...
    'NumPoints', 1, 'ReadKey', 'YOUR_CHANNEL_READ_KEY');
if numel(v) == 3 && v(3) == 1
    options = weboptions('HeaderFields', ...
        {'ThingSpeak-Alerts-API-Key', 'YOUR_ALERTS_API_KEY'});
    webwrite('https://api.thingspeak.com/alerts/send', ...
        'subject', 'Weather Station Alert', ...
        'body', sprintf(['Temperature %.1f C; humidity %.1f percent. ' ...
        'Limits: T >35 C or RH >80 percent.'], v(1), v(2)), options);
end
```

Threshold interpretation

The comparisons are strict: a value of exactly 35 C or 80% RH does not independently raise an alarm. The alarm is 1 when either value exceeds its limit. The received temperature and humidity messages demonstrate each branch separately; boundary values were not separately measured during this test.

Submission folder

Upload the IoT_Weather_Station_Haresh folder to Google Drive and submit the folder sharing link required by the assignment. It contains this PDF, source files, the simulator import archive and the screenshots. No Google Drive link is embedded in this report.